# Backend Environment Set Up Doc

## This document captures the complete backend environment setup for the Budget App using pnpm, NestJS, PostgreSQL, and Prisma. It is written to be reproducible, production-oriented, and suitable for future onboarding.

---

## 1. Prerequisites

Ensure the following are installed on your system:

- **Node.js (LTS, v20+)**
- **pnpm (latest)**
- **Git**
- **PostgreSQL**
- **pgAdmin / TablePlus (DB GUI)**
- **VS Code**

Verify:

```
node -v
pnpm -v
git --version
psql --version
```

---

## 2. Project Creation (pnpm-based)

Create NestJS backend using pnpm:

```
nest new budget-app-backend --package-manager=pnpm
cd budget-app-backend
pnpm start:dev
```

If `http://localhost:3000` responds, the backend runtime is alive.

---

# 3. Package Manager Rule

> One project → One package manager → One lock file

- Use **pnpm only**
- Do not mix with npm or yarn
- Commit `pnpm-lock.yaml`

---

# 4. Environment Variables Setup

## 4.1 Install Config Support

```
pnpm add @nestjs/config
```

## 4.2 Create `.env`

```
PORT=3000
DATABASE_URL="postgresql://postgres:<password>@localhost:543
2/budget_db"
JWT_SECRET="dev-super-secret-key"
JWT_EXPIRES_IN="7d"
```

## 4.3 Protect Secrets

`.gitignore`

```
.env
```

```
.env.*
```

### 4.4 Load Env in NestJS

`src/app.module.ts`

```
import { ConfigModule } from '@nestjs/config';

ConfigModule.forRoot({
  isGlobal: true,
});
```

### 4.5 Validate Env (Optional but Recommended)

Install Zod:

```
pnpm add zod
```

Create `src/config/env.schema.ts` and wire it using `validate()`.

---

## 5. Database Setup (PostgreSQL + Prisma)

### 5.1 Install Prisma (Pinned Stable Version)

```
pnpm add -D prisma@5.10.2
pnpm add @prisma/client@5.10.2
```

### 5.2 Initialize Prisma

```
pnpm prisma init
```

Creates:

- `prisma/schema.prisma`
- `.env` reference

---

## 5.3 Configure Prisma Schema

`prisma/schema.prisma`

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

---

## 5.4 Create Database

Using pgAdmin / SQL:

```
CREATE DATABASE budget_db;
```

Validate:

```
pnpm prisma validate
```

---

## 5.5 Initialize Migrations

```
pnpm prisma migrate dev --name init
```

Creates `_prisma_migrations` table and validates DB connection.

---

## 5.6 Add First Model (User)

Append to `schema.prisma`:

```
model User {
  id        String   @id @default(uuid())
```

```
  email     String   @unique
  password  String
  name      String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

### Design Notes

- UUID primary key (API-safe)
- Email uniqueness enforced at DB level
- Nullable name for UX flexibility
- Automatic timestamps for auditing

---

## 5.7 Apply Migration

```
pnpm prisma migrate dev --name add_user
```

Results:

- SQL migration generated
- Table created in Postgres
- Prisma Client regenerated

---

## 5.8 Verify Database

### pgAdmin

- Refresh DB
- Navigate: `Schemas → public → Tables`
- Confirm:
  - `User`

- `_prisma_migrations`

### Prisma Studio (Optional)

```
pnpm prisma studio
```

---

## 6. ESLint (Modern Flat Config)

### Why this section exists

NestJS (latest versions) uses **ESLint Flat Config**. During setup, you may encounter a **deprecation warning** related to `tseslint.config(...)`. This project explicitly avoids deprecated helpers and uses the **ESLint core standard** instead.

> `tseslint.config(...)` is deprecated because ESLint core now provides the same functionality via `defineConfig()`.

This documentation reflects the **future-proof, recommended approach**.

---

### 6.1 Install ESLint Dependencies

```
pnpm add -D eslint @eslint/js typescript-eslint eslint-plugin
-prettier globals
```

---

### 6.2 ESLint Configuration (Non-deprecated)

`eslint.config.mjs`

```
// @ts-check
import { defineConfig } from 'eslint/config';
import eslint from '@eslint/js';
import tseslint from 'typescript-eslint';
import eslintPluginPrettierRecommended from 'eslint-plugin-pr
ettier/recommended';
import globals from 'globals';
```

```
export default defineConfig([
  {
    ignores: ['eslint.config.mjs'],
  },

  eslint.configs.recommended,

  ...tseslint.configs.recommendedTypeChecked,

  eslintPluginPrettierRecommended,

  {
    languageOptions: {
      globals: {
        ...globals.node,
        ...globals.jest,
      },
      sourceType: 'commonjs',
      parserOptions: {
        projectService: true,
        tsconfigRootDir: import.meta.dirname,
      },
    },
  },

  {
    rules: {
      '@typescript-eslint/no-explicit-any': 'off',
      '@typescript-eslint/no-floating-promises': 'warn',
      '@typescript-eslint/no-unsafe-argument': 'warn',
      'prettier/prettier': ['error', { endOfLine: 'auto' }],
    },
  },
]);
```

### 6.3 Notes on Deprecation

- ❌ Do **not** use `tseslint.config(...)`
- ✅ Use `defineConfig()` from `eslint/config`
- This removes TypeScript deprecation warnings
- Aligns with ESLint core direction (2025+)

---

## 7. Prisma Integration with NestJS

This section documents how Prisma is **properly integrated into NestJS** using Dependency Injection and lifecycle hooks. This ensures safe database connections, testability, and production readiness.

---

### 7.1 Why Prisma Needs a NestJS Module

Prisma Client **must not** be instantiated manually (`new PrismaClient()`) across the codebase.

Using a dedicated Prisma module gives:

- Single shared DB connection
- Clean lifecycle management
- Centralized infrastructure boundary
- Easy mocking for tests
- Future extensibility (middlewares, multi-tenancy)

This follows NestJS + Clean Architecture best practices.

---

### 7.2 Create Prisma Module Structure

Create a new folder:

```
src/
├── prisma/
│   ├── prisma.module.ts
```

```
│   └── prisma.service.ts
```

## 7.3 PrismaService (Lifecycle-Aware Client)

`src/prisma/prisma.service.ts`

```
import {Injectable,OnModuleInit,OnModuleDestroy }from'@nestj
s/common';
import {PrismaClient }from'@prisma/client';

@Injectable()
exportclassPrismaService
extendsPrismaClient
implementsOnModuleInit,OnModuleDestroy
{
asynconModuleInit() {
awaitthis.$connect();
  }

asynconModuleDestroy() {
awaitthis.$disconnect();
  }
}
```

## Design Rationale

- `extends PrismaClient`

  → Exposes full Prisma API without wrappers

- `OnModuleInit`

  → DB connects when NestJS boots

- `OnModuleDestroy`

  → Clean shutdown for graceful exits and tests

This aligns Prisma with NestJS's application lifecycle.

---

## 7.4 PrismaModule (Global Infrastructure Module)

`src/prisma/prisma.module.ts`

```
import {Global,Module }from'@nestjs/common';
import {PrismaService }from'./prisma.service';

@Global()
@Module({
providers: [PrismaService],
exports: [PrismaService],
})
exportclassPrismaModule {}
```

### Key Decisions

- `@Global()`

  Prisma is infrastructure, not domain logic.

  This avoids repetitive imports across modules.

- `exports: [PrismaService]`

  Makes Prisma available via DI everywhere.

---

## 7.5 Register PrismaModule in AppModule

`src/app.module.ts`

```
import {Module }from'@nestjs/common';
import {ConfigModule }from'@nestjs/config';
import {PrismaModule }from'./prisma/prisma.module';

@Module({
imports: [
ConfigModule.forRoot({isGlobal:true }),
```

```
PrismaModule,
  ],
})
exportclassAppModule {}
```

At runtime:

1. NestJS boots
2. Environment variables load
3. Prisma connects to PostgreSQL
4. Application becomes ready to serve requests

---

### 7.6 Verify Prisma–Nest Integration

Start the server:

```
pnpm start:dev
```

If no Prisma connection errors appear, the integration is successful.

### 7.7 Verify Prisma–Nest Integration

At this stage, the backend has:

- Lifecycle-managed Prisma Client
- Global database access via NestJS DI
- Clean infrastructure boundary
- Production-safe DB integration

This completes **environment + database wiring**.

---

## 8. Current State Summary

At this point, the backend has:

- pnpm-based dependency management
- NestJS runtime
- Environment variable handling
- PostgreSQL database
- Prisma ORM with migrations
- First domain model (`User`)
- Modern ESLint setup
- PrismaModule & PrismaService integration

This setup is **production-grade and scalable**.

---

*End of Environment Setup Documentation.*